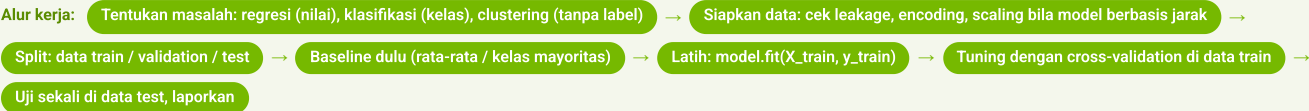


Machine Learning Fundamentals

Ringkasan 1 halaman untuk hari 1: alur kerja ML, metrik, regresi & time series, klasifikasi, ensemble, clustering, dan kapan GPU membantu. Dipakai bersama 10 notebook Colab.



① Cara model belajar & diuji

- Feature = kolom masukan; target = yang diprediksi
- Belajar = mengubah parameter sampai loss (total error) sekecil mungkin
- Data train untuk melatih, validation untuk memilih model, test disimpan untuk penilaian akhir
- Leakage: memakai informasi yang belum tersedia saat prediksi (contoh: lama panggilan, baru diketahui setelah telepon selesai)
- Underfitting = terlalu sederhana; overfitting = mengikuti noise. Cek gap train vs validation

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)
cross_val_score(model, X_tr, y_tr, cv=5).mean()
```

► Hyperparameter dipilih lewat CV, bukan dari data test

② Metrik

- **Regresi:** MAE rata-rata [error]; RMSE lebih peka ke error besar (dikuadratkan); R²: 1 = prediksi tepat di data itu, 0 = setara rata-rata, negatif = lebih buruk; MAPE dalam persen
- **Klasifikasi:** positif = kondisi yang mau dideteksi. Precision = dari yang diprediksi positif, berapa benar. Recall = dari yang benar-benar positif, berapa tertangkap. F1 = rata-rata harmonik keduanya
- ROC-AUC: peluang model mengurutkan positif di atas negatif; 0,5 = tebak acak
- Baseline dulu: akurasi 89% di data yang 89% negatif = tidak menambah apa pun

► Threshold menggeser trade-off precision vs recall

③ Linear regression

- $\hat{y} = w \cdot x + b$; w = kemiringan (pengaruh satu satuan x); b = intercept, prediksi saat x = 0
- Koefisien besar ≠ sebab-akibat
- Regularisasi (Ridge/Lasso) menambah penalti ukuran koefisien; Lasso bisa menolak feature yang tidak membantu

```
LinearRegression().fit(X_tr, y_tr)
Lasso(alpha=1.0).fit(scaler.fit_transform(X_tr),
y_tr)
```

► Target berupa nilai kontinu, hubungan mendekati linear

④ Time series (SARIMA)

- Komponen: trend, pola musiman, noise; urutan waktu penting
- Split kronologis: train bulan awal, test 12 bulan terakhir
- Stasioner = sifat statistik stabil; differencing (selisih antar periode) biasanya membantu; stasioneritasnya tetap diuji
- SARIMA: AR (nilai sebelumnya), I (differencing), MA (error sebelumnya), S (musiman 12)
- Baseline seasonal naive: bulan ini = bulan yang sama tahun lalu

```
auto_arima(train, seasonal=True,
m=12).predict(12, return_conf_int=True)
```

► Laporkan MAPE vs baseline dan interval prediksi

⑤ Logistic regression

- Skor → sigmoid → probabilitas 0–1; threshold 0,5 default
- Data timpang: di modul ini class_weight='balanced' + geser threshold, bukan oversampling
- Batas keputusan garis lurus; perlu scaling; mudah dijelaskan

```
LogisticRegression(class_weight='balanced',
max_iter=1000)
(model.predict_proba(X)[: ,1] >= 0.4)
```

► Klasifikasi biner yang butuh probabilitas

⑥ Decision tree · KNN · SVM

- **Tree:** tiap simpul membayangkan satu feature dengan ambang (gini); batas sejajar sumbu feature; tanpa scaling; batasi depth lewat CV
- **KNN:** mayoritas dari K tetangga terdekat (jarak Euclidean); K kecil overfit, K besar underfit; scaling bila skala feature berbeda; prediksi melambat saat data besar
- **SVM:** cari margin terlebar; support vector menentukan batas; C = ketat/longgar margin; kernel RBF untuk batas melengkung; perlu scaling

```
DecisionTreeClassifier(max_depth=best_depth)
KNeighborsClassifier(n_neighbors=best_k)
SVC(kernel='rbf', C=1.0)
```

► Bandingkan boundary-nya di dataset yang sama

⑦ Ensemble

- Gabungan membantu kalau kesalahan anggotanya tidak seragam
- Voting: hard (1 suara/model) atau soft (rata-rata probabilitas)
- Bagging/Random Forest: bootstrap = sampel dengan pengembalian (satu baris boleh terpilih lebih dari sekali) + subset feature acak; menurunkan variance
- Boosting: model berurutan memperbaiki kesalahan sebelumnya; menurunkan bias
- Stacking: meta-model memutuskan dari prediksi model dasar
- No free lunch: selalu bandingkan beberapa model

```
RandomForestClassifier(n_estimators=500,
random_state=42)
```

► Pilihan awal yang aman untuk data tabular

⑧ XGBoost

- Gradient boosting yang cepat, regularisasi bawaan, tangani nilai kosong
- Early stopping: berhenti saat skor validation tidak membaik
- scale_pos_weight untuk kelas timpang
- Feature importance = kontribusi menurut metode, bukan sebab-akibat
- Encoding dummy ditentukan dari data train, test disamakan (reindex)

```
XGBClassifier(n_estimators=1000,
early_stopping_rounds=50,
eval_metric='auc').fit(X_tr, y_tr, eval_set=
[(X_val, y_val)])
```

► Sering menang di kompetisi data tabular

⑨ Clustering

- K-means: tiap titik ke pusat terdekat → hitung ulang rata-rata → ulangi; titik diam, keanggotaan & pusat berubah
- Pilih K: elbow (inertia) & silhouette (–1..1, makin tinggi makin rapi)
- Hierarchical: gabungkan dua kelompok terdekat berulang; potong dendrogram
- DBSCAN: berbasis kepadatan, eps dari k-distance plot, deteksi outlier (pencilan)
- Jebakan: kolom kategori ikut → cluster meniru kolom itu; data miring → log1p dulu

```
X =
StandardScaler().fit_transform(np.log1p(data[spen
d_cols]))
KMeans(n_clusters=k, n_init=10,
random_state=42).fit_predict(X)
```

► Cek hasil dengan cross-tab ke label yang tidak dipakai

⑩ Kapan GPU membantu

- GPU unggul pada operasi paralel besar: ribuan pohon, jarak ke banyak titik, matriks besar
- Tidak menang di data kecil: overhead transfer & warm-up melebihi kerja hitungannya. Ukur setelah warm-up
- cuML: API mirip scikit-learn, ganti import saja; tidak semua argumennya tersedia
- Hasil Colab T4, Covertipe 581 ribu baris: RF 16x, XGBoost 9x, KNN 22x, K-Means 3,5x; PCA 0,2x (GPU kalah, operasinya terlalu kecil)
- XGBoost/KNN skor identik di CPU dan GPU; Random Forest cuML bisa beda sedikit (implementasi berbeda), cek ulang skornya

```
from cuml.ensemble import RandomForestClassifier
XGBClassifier(device='cuda')
```

► Sebut syarat pengukurannya: tugas, ukuran data, perangkat, setelah warm-up

Kapan pakai model yang mana

Model	Bentuk batas keputusan	Scaling?	Kecepatan	Interpretasi	Catatan
Logistic regression	Garis lurus	Ya	Cepat	Mudah	Butuh probabilitas; kelas timpang → class_weight
Decision tree	Sejajar sumbu feature	Tidak	Cepat	Mudah	Batasi depth lewat CV
KNN	Mengikuti sebaran data	Ya (bila skala beda)	Lambat saat prediksi	Sulit	Data kecil, sedikit feature
SVM RBF	Melengkung	Ya	Lambat di data besar	Sulit	Atur C dan gamma
Random Forest	Fleksibel	Tidak	Sedang (paralel)	Sedang (feature importance)	Pilihan awal yang aman
XGBoost	Fleksibel	Tidak	Cepat	Sedang	Pakai early stopping

Isilah kunci

Leakage — model memakai informasi yang belum tersedia saat prediksi; evaluasi jadi terlalu optimistik.

Baseline — pembandingan paling sederhana (rata-rata; kelas mayoritas; seasonal naive). Model layak kalau melampauinya.

Cross-validation — bagi data train jadi k fold, latih/uji bergiliran, ambil rata-rata; untuk memilih hyperparameter tanpa menyentuh test.

Intercept — nilai prediksi saat semua feature nol; sering di luar rentang data, jadi jangan ditafsirkan berlebihan.

Bias–variance — model terlalu sederhana melewatkan pola (bias); model terlalu peka berubah banyak saat data train berubah (variance).

Bootstrap — sampel acak dengan pengembalian dari data yang ada; dasar bagging dan Random Forest.

Interval prediksi — rentang untuk nilai baru dengan tingkat cakupan tertentu (mis. 95%); bukan jaminan.

Outlier (pencilan) — titik yang jauh dari kelompok mana pun; belum tentu data salah.